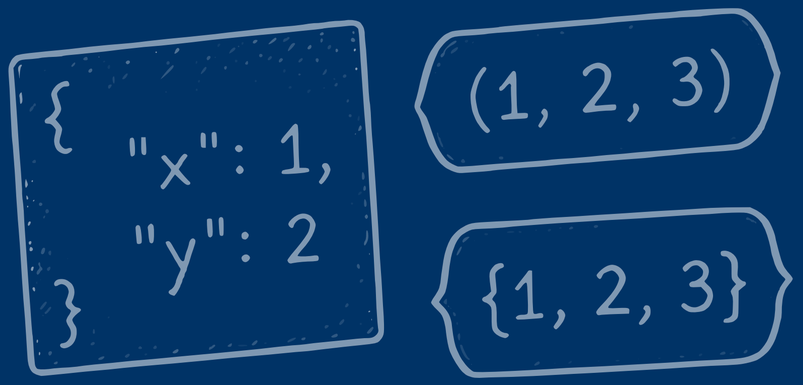


Programming

Lecture 07 — Collections

Tuples, sets and dictionaries. Choosing how to hold many values

Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27

UNIVERSITÀ DEGLI STUDI DI MILANO

INFN MILANO

Where we left off

Lecture 06, in four lines

A list is a mutable sequence

Indexing and slicing as on strings, and you can change it.

= never copies

b = a is a second name for the same list.

enumerate and zip

Positions and items together. Two lists in step.

Tables are lists of lists

m[row][col], row first, always.

A list answers one question very well. What is at position 3? Today: three structures for the questions a list answers badly.

Three promises come due. The pair in for i, g in enumerate(...). The parallel lists that break when one is sorted. Lecture 5's letter count, in one pass.

By the end of today you can…

- group values that belong together, and take them apart again
- remove duplicates, and compare two collections, with sets
- look things up by name with a dictionary
- count things in a single pass
- say which structure fits a problem, and why

Carry one question through the lecture. What will I ask this data? The answer picks the structure.

A

Tuples and sets

Fixed groups · distinct values · fast membership

A tuple is a fixed group of values

One thing made of several parts

```
point   = (3, 4)
date    = (2026, 10, 23)
student = ("Ada", 1815, "London")

point[0]        ->  3
len(date)       ->  3
student[-1]     ->  "London"
```

```
point[0] = 9   ->  TypeError
```

A list

Many things of the same kind. Grades, names, temperatures. It grows and shrinks, and its length tells you how many.

A tuple

One thing with several parts. Each position has its own meaning: x then y, or year, month, day. The number of parts is fixed by what the thing is: a point in 2D has exactly two.

Indexing, slicing, len and in work exactly as on lists. Changing does not. A tuple is immutable, like a string.

Packing and unpacking

The comma makes the tuple

```
p = 3, 4          # packing
x, y = p          # unpacking

print(x, y)       # 3 4
```

```
a, b, c = (1, 2)  ->  ValueError: not enough values
                      to unpack (expected 3, got 2)
```

The swap, in one line

```
a, b = b, a
```

Right-hand side first, as always since lecture 2. Python packs (b, a) into a tuple, then unpacks it into a and b. No temporary variable. Lecture 6 needed three lines for this in selection sort.

(5) is just the number 5 in brackets. A tuple with one item needs a comma, (5,). The brackets only group, the comma is what makes a tuple.

The number of names on the left must match the number of parts on the right, exactly.

You have used tuples already

Lecture 6 kept a promise for today

```
list(enumerate(["a", "b"]))          ->  [(0, "a"), (1, "b")]
list(zip(["Ada", "Alan"], [30, 27])) ->  [("Ada", 30), ("Alan", 27)]
divmod(17, 5)                        ->  (3, 2)
```

`for i, g in enumerate(grades)`	enumerate hands over a tuple (i, g) each time round. The loop unpacks it into two names. That is the pair from lecture 6.
`for name, g in zip(names, grades)`	Same mechanism. zip builds the tuples, the loop takes them apart.
`q, r = divmod(17, 5)`	A function that gives back two answers gives back one tuple. Next week you write functions like that yourself.

Why have a list you cannot change?

Three good reasons

Safe to share

Like a string, nobody can change it behind your back. Pass it anywhere, keep it as long as you like.

The shape means something

A date has three parts. A fourth part would be a bug, and a tuple keeps it out.

It can be a key

Sets and dictionaries only accept values that cannot change. A tuple qualifies, a list does not. Why, in two slides.

A bonus: tuples compare part by part

```
(2026, 10, 23) < (2026, 11, 1)    ->  True     years equal, so compare months
sorted([(2026, 11, 1), (2025, 12, 24), (2026, 10, 23)]) ->  oldest date first
```

Like strings, which compare character by character. Store dates as (year, month, day) and they sort correctly with no extra work.

A set is a collection of distinct values

No duplicates, no order

```
s = {3, 1, 3, 2}
print(s)            # {1, 2, 3}

len(s)              ->  3
2 in s              ->  True
set("banana")       ->  {"a", "b", "n"}
```

```
s[0]  ->  TypeError: 'set' object is not subscriptable
```

Changing a set

```
s.add(7)
s.remove(3)
```

add ignores a value that is already there. remove raises KeyError for one that is not.

The empty set is set(). Plain {} is already taken by the empty dictionary, part B.

A set has no positions, so no indexing and no slicing. Small numbers often print in order. That is a coincidence, and strings will not do it.

Membership is what sets are for

Why x in s is fast

x in a list



Compare with the first item, then the second, then the third. That is linear search from lecture 6: up to n comparisons.

x in a set



The value itself decides where it is stored, like a cloakroom ticket that tells you the hook. So Python looks in one place, whatever the size.

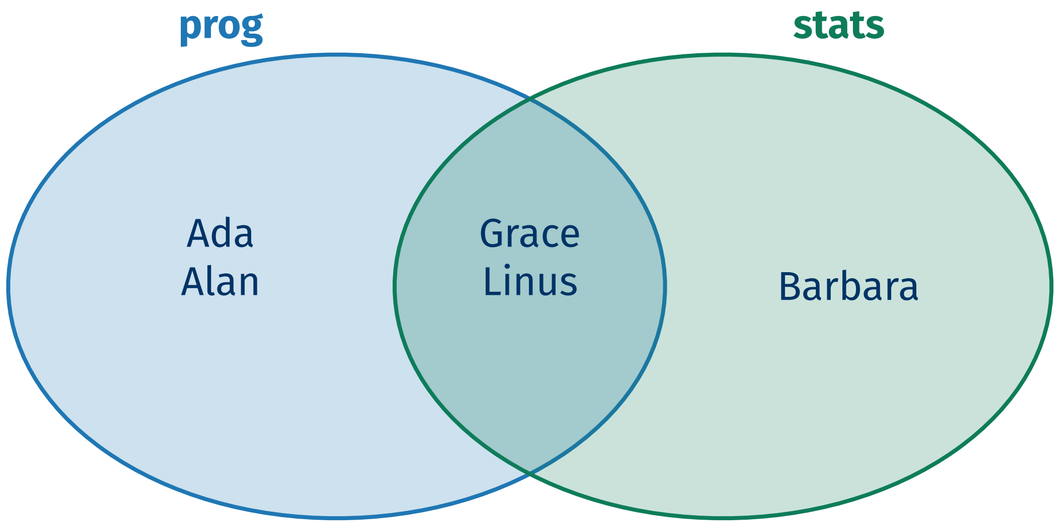
items	x in a list	x in a set
1 000	up to 1 000 looks	about 1
1 000 000	up to 1 000 000 looks	about 1

This is why set items must be immutable. A value that changed after it was stored would hang on the wrong hook. {[1, 2]} raises TypeError: unhashable type.

Comparing two collections

Set operations

```
prog  = {"Ada", "Alan", "Grace", "Linus"}
stats = {"Grace", "Linus", "Barbara"}
```



`prog & stats`	**in both**	{"Grace", "Linus"}
`prog | stats`	**in either**	all five names
`prog - stats`	**only in prog**	{"Ada", "Alan"}
`prog ^ stats`	**in exactly one**	Ada, Alan, Barbara

Each of these would be a nested loop over two lists. With sets it is one operator, and it reads like the question you are asking.

Three structures so far

Before dictionaries

	list [ ]	**tuple ( )**	**set { }**
keeps an order	**yes**	**yes**	no
can be changed	**yes**	no	**yes**
allows duplicates	**yes**	**yes**	no
x[i] works	**yes**	**yes**	no
x in it is fast	no	no	**yes**
the question it answers	what is at position i?	what are the parts of this one thing?	is x in there?

The last row is the one to remember. The others follow from it.

Worked example

Who has not handed in?

What you have

The class list: eight students.

The names typed on thirteen submissions, exactly as the students typed them.

What you want

- How many students handed in.
- Who has not handed in.
- Which submitted names are not on the class list at all.

The data is messy on purpose. Some students submitted twice, some typed their name in lower case, some left spaces around it, and one made a typo.

In the notebook, part A.

Exercises — Block A

Sheet 07, part A · 40 minutes · predict before you run

- Predict what eight lines do with tuples, including two that fail.
- Predict what eight lines do with sets.
- Choose a structure for six situations, and justify each choice.
- Find the letters a sentence never uses.
- Work with a list of points stored as tuples.

For every "which structure" question, first write the question you will ask the data. Then choose.

B

Dictionaries

Look up by key · count in one pass · records

Looking something up by name

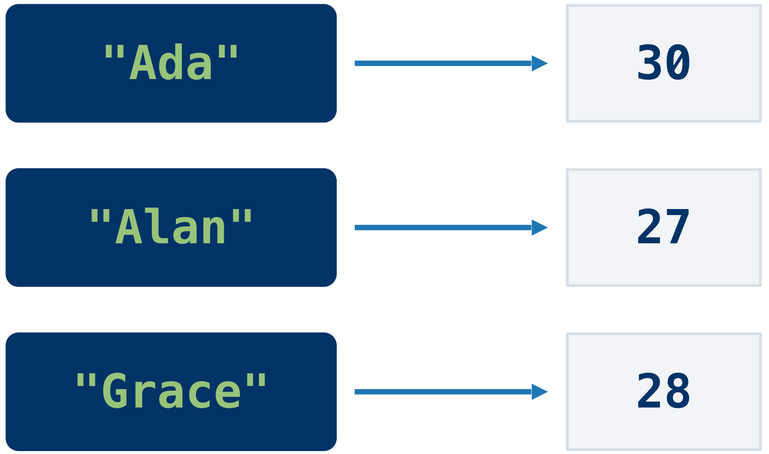
What parallel lists cannot do

```
names  = ["Ada", "Alan", "Grace"]
grades = [30, 27, 28]

i = names.index("Grace")      # a search
print(grades[i])              # 28
```

- Finding Grace means walking the list of names.
- Sort one list and every pair is wrong, silently.
- Add a student and you must remember to add to both.

What we want



Each name attached to its grade. Ask "Grace?" and get 28, with no searching and nothing to keep in step.

A dictionary maps keys to values

Look up by key, not by position

```
grades = {"Ada": 30, "Alan": 27, "Grace": 28}

grades["Grace"]       ->  28
len(grades)           ->  3
"Ada" in grades       ->  True
30 in grades          ->  False     in looks at the keys
```

Keys are unique

Write the same key twice and the last value wins. A key is a name, and a name means one thing.

Keys cannot change

Strings, numbers and tuples work. Lists and sets ***do not!***

Values can be anything

Numbers, strings, lists, other dictionaries. Duplicates among values are fine.

Looking up a key is as fast as set membership, whatever the size.
A dictionary is a set of keys with a value hanging on each hook.

Changing a dictionary

One syntax adds and replaces

```
grades["Barbara"] = 29      # a new key: added
grades["Alan"] = 28         # an existing key: replaced
del grades["Ada"]           # removed
g = grades.pop("Grace")     # removed, and gives back 28
```

d[key] = value never asks. If the key exists, the old value is gone. There is no append for dictionaries, and no need for one.

Dictionaries are mutable

Everything from lecture 6 about names and boxes holds. d2 = d gives a second name for the same dictionary, and a change through one shows through the other. Copy with d.copy().

The missing key

Three ways to ask safely

```
grades["Linus"]   ->  KeyError: 'Linus'
```

Check first

```
if "Linus" in grades:
    print(grades["Linus"])
```

When a missing key needs its own branch, like a message to the user.

Ask with get

```
grades.get("Linus")
                  ->  None
```

When "nothing" is a fine answer and you test for None later.

Ask with a default

```
grades.get("Linus", 0)
                  ->  0
```

When a missing key has a natural value. This one is the key to counting.

None of the three is "the right one". Choose by what should happen when the key is not there.

Walking a dictionary

Keys, values, or both

```
for name in grades:                 # keys
    print(name)

for g in grades.values():           # values
    print(g)

for name, g in grades.items():      # both
    print(f"{name:<6} {g}")
```

- A plain for gives the keys.
- .items() gives tuples, and the loop unpacks them. Block A again.
- The order is the order the keys were added.
- sorted(grades) gives the keys in alphabetical order.

Do not add or remove keys while you walk a dictionary. Python stops you with RuntimeError. Collect what to change first, change it after the loop.

The counting pattern

One pass, any number of categories

```
counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1

print(counts)    # {"b": 1, "a": 3, "n": 2}
```

ch	counts.get(ch, 0)	new count
"b"	0	1
"a"	0	1
"n"	0	1
"a"	1	2
"n"	1	2
"a"	2	3

Read the line right to left. How many so far, or 0 if this is the first one. Add one. Store it back under the same key.

Lecture 5 counted letters by asking the text 26 times, once per letter. This reads the text once, and only makes entries for what actually appears.

Values can be lists and dictionaries

Records and groups

A dictionary of lists: one group per key

```
exams = {"Ada": [30, 28, 29],
         "Alan": [27]}

exams["Ada"][1]           ->  28
exams["Alan"].append(25)
```

Read the chain left to right, as with m[row][col]. exams["Ada"] is a list, so [1] picks from that list.

A dictionary as a record: named parts

```
ada = {"name": "Ada",
       "born": 1815,
       "city": "London"}

ada["city"]    ->  "London"
```

Compare with the tuple ("Ada", 1815, "London"). ada["city"] says what it means, ada_tuple[2] does not.

Most real data looks like this: lists of records, records with lists inside. Each level is chosen by the question you ask at that level.

Choosing a structure

Start from the question you will ask

What is at position i? Keep this order.	`list`	*grades in the order they were handed in*
These parts belong together, and the shape is fixed.	`tuple`	*a position on a map, (latitude, longitude)*
Is x in there? Each value once.	`set`	*the students who handed in*
What belongs to this key?	`dict`	*student name to grade*

Combine them freely. A dictionary of lists, a list of tuples, a set of tuples. When in doubt, write down the question first.

Worked example

Which words does a text use most?

What you have

One paragraph of English text, pasted into a string.

Capitals, commas and full stops are stuck to the words.

What you want

- How many words, and how many different words.
- The five most frequent words, with their counts.
- The same again, without the small words: the, of, and, a, …

Before opening the notebook, decide which structure from today answers each line on the right.

In the notebook, part B.

Exercises — Block B

Sheet 07, part B · 40 minutes · plan in comments first

Read and predict

- Ten lines on one dictionary.
- A counter that crashes on its first item.

Write

- Count votes and find the winner, ties included.
- Translate word by word, keeping unknown words.
- Turn a dictionary round, and group by value.
- Query a small table of student records.

Before writing a loop, say in one sentence what the dictionary holds at the end. For example, keys are names and values are vote counts.

Today in one slide

A tuple is one thing in parts

(3, 4). Immutable. The comma makes it, unpacking takes it apart.

a, b = b, a

Right side packed first, then unpacked. The swap in one line.

A set holds distinct values

No order, no duplicates, and x in s is fast at any size.

Set operations

& both, | either, - only the first, ^ exactly one.

A dictionary maps keys to values

d[key] reads and writes. get(key, default) for keys that may be missing.

Counting in one pass

counts[x] = counts.get(x, 0) + 1

Before next time

- Finish sheet 07. Solutions go up after we discuss them.
- Read sections 5.3 to 5.5 of the official Python tutorial: tuples, sets and dictionaries.
- Rewrite one program from an earlier sheet that used two parallel lists, with a dictionary.
- Bring the laptop.

Next: Lecture 08 — functions. You have called them since week one. Next week you write your own, and key=str.lower finally makes sense.